

fairyland_dashboard

March 7, 2026

1 FAIRyland SPARQL Dashboard Visualization

This notebook creates a comprehensive summary dashboard from the FAIRyland RDF dataset. It demonstrates SPARQL queries for archaeological data and generates publication-ready visualizations.

The dashboard includes: - Feature type inventory (filtered and cleaned) - Stratigraphic sequence distribution (log scale) - Spatial distribution by excavation trench - Preservation state analysis (Kötbullar)

Dataset: ../lod/fairyland.ttl

Author: Florian Thiery, Research Squirrel Engineers

License: CC-BY 4.0

Project: <https://github.com/Research-Squirrel-Engineers/FAIRyland>

1.1 Setup & Imports

```
[1]: # Import required libraries
import rdflib
from rdflib import Graph, Namespace
import pandas as pd
import matplotlib.pyplot as plt
import os

# Jupyter-specific: better plotting in notebooks
%matplotlib inline
plt.rcParams['figure.dpi'] = 100 # Good quality for notebook display
```

1.2 SPARQL Queries

Define the SPARQL queries used to extract data from the RDF graph.

```
[2]: DASHBOARD_QUERIES = {
    "query1_feature_inventory": {
        "title": "Feature Type Inventory",
        "description": "Count archaeological features by type",
        "sparql": ""PREFIX fairyland: <https://github.com/
↳Research-Squirrel-Engineers/FAIRyland/>
```

```

SELECT ?type (COUNT(?feature) AS ?count)
WHERE {
  ?feature a ?type .
  FILTER(STRSTARTS(STR(?type), "https://github.com/Research-Squirrel-Engineers/
↳FAIRyland/"))
}
GROUP BY ?type
ORDER BY DESC(?count)""",
  },
  "query2_koetbullar": {
    "title": "Kötbullar Preservation Analysis",
    "description": "Analyze preservation state of archaeological features",
    "sparql": ""PREFIX fairyland: <https://github.com/
↳Research-Squirrel-Engineers/FAIRyland/>
PREFIX suni: <http://www.github.com/sparqlunicorn#>

SELECT ?condition (COUNT(?koetbullar) AS ?count)
WHERE {
  ?koetbullar a fairyland:Koetbullar .
  OPTIONAL { ?koetbullar suni:Condition ?condition }
}
GROUP BY ?condition
ORDER BY DESC(?count)""",
  },
  "query3_stratigraphy": {
    "title": "Stratigraphic Distribution",
    "description": "Distribution of features across time periods",
    "sparql": ""PREFIX suni: <http://www.github.com/sparqlunicorn#>

SELECT ?period (COUNT(?feature) AS ?count)
WHERE {
  ?feature suni:Time_Period ?period .
}
GROUP BY ?period
ORDER BY ?period""",
  },
  "query4_trench": {
    "title": "Spatial Distribution",
    "description": "Feature counts by excavation trench",
    "sparql": ""PREFIX suni: <http://www.github.com/sparqlunicorn#>

SELECT ?trench (COUNT(?feature) AS ?count)
WHERE {
  ?feature suni:Trench ?trench .
}
GROUP BY ?trench
ORDER BY DESC(?count)""",

```

```
    },
}

print(" Queries defined")
```

Queries defined

1.3 Load RDF Dataset

Load the FAIRyland dataset from Turtle format.

```
[3]: # Determine path to TTL file
# Adjust this path if your notebook is in a different location!
ttl_path = "../lod/fairyland.ttl"

# Alternative: Use absolute path
# ttl_path = "/path/to/your/fairyland.ttl"

print(f"Loading: {ttl_path}")

g = Graph()
g.parse(ttl_path, format="turtle")

print(f" Loaded {len(g):,} triples")
```

```
Loading: ../lod/fairyland.ttl
Loaded 1,847 triples
Loaded 1,847 triples
```

1.4 Execute SPARQL Queries

Run all queries and store results in pandas DataFrames.

```
[4]: results = {}

# Query 1: Feature Inventory
print("→ Query 1: Feature Type Inventory")
df1 = pd.DataFrame(
    g.query(DASHBOARD_QUERIES["query1_feature_inventory"]["sparql"]),
    columns=["type", "count"],
)
df1["type"] = df1["type"].apply(lambda x: str(x).split("/)[-1])
df1["count"] = df1["count"].astype(int)
results["inventory"] = df1
print(f" Found {len(df1)} feature types")

# Query 2: Kötbullar Condition
print("→ Query 2: Preservation Analysis")
df2 = pd.DataFrame(
```

```

    g.query(DASHBOARD_QUERIES["query2_koetbullar"]["sparql"]),
    columns=["condition", "count"],
)
df2["condition"] = df2["condition"].apply(lambda x: str(x) if x else "intact")
df2["count"] = df2["count"].astype(int)
results["koetbullar"] = df2
print(f" Total features analyzed: {df2['count'].sum()}")

# Query 3: Stratigraphy
print("→ Query 3: Stratigraphic Distribution")
df3 = pd.DataFrame(
    g.query(DASHBOARD_QUERIES["query3_stratigraphy"]["sparql"]),
    columns=["period", "count"],
)
df3["count"] = df3["count"].astype(int)
results["stratigraphy"] = df3
print(f" Found {len(df3)} time periods")

# Query 4: Trench Distribution
print("→ Query 4: Spatial Distribution")
df4 = pd.DataFrame(
    g.query(DASHBOARD_QUERIES["query4_trench"]["sparql"]),
    columns=["trench", "count"],
)
df4["count"] = df4["count"].astype(int)
results["trench"] = df4
print(f" Found {len(df4)} trenches")

print("\n All queries executed successfully")

```

```

→ Query 1: Feature Type Inventory
  Found 9 feature types
→ Query 2: Preservation Analysis
  Total features analyzed: 136
→ Query 3: Stratigraphic Distribution
  Found 4 time periods
→ Query 4: Spatial Distribution
  Found 2 trenches

```

All queries executed successfully

1.5 Inspect Query Results

Let's look at the raw data before visualization.

```

[5]: # Display results
print("Feature Inventory:")
display(results["inventory"])

```

```

print("\nStratigraphy:")
display(results["stratigraphy"])

print("\nTrench Distribution:")
display(results["trench"])

print("\nKötbullar Preservation:")
display(results["koetbullar"])

```

Feature Inventory:

	type	count
0	Koetbullar	136
1	Minion	18
2	Street	7
3	TimePeriod	7
4	Stone	4
5	Kangaroo	4
6	Crater	2
7	Trench	2
8	AllenKey	1

Stratigraphy:

	period	count
0	Minion	164
1	Minion I	1
2	Minion IIa	5
3	Minion IIb	2

Trench Distribution:

	trench	count
0	Ikea Land	164
1	Norrfors	8

Kötbullar Preservation:

	condition	count
0	intact	118
1	smashed	18

1.6 Create Dashboard Visualization

Generate the complete 2×2 dashboard with all four panels.

```

[6]: fig = plt.figure(figsize=(20, 12))
gs = fig.add_gridspec(2, 2, hspace=0.35, wspace=0.3)

# =====
# Panel 1: Feature Type Inventory
# =====

ax1 = fig.add_subplot(gs[0, 0])

# Filter out metadata types (TimePeriod, Trench)
df_filtered = results["inventory"][
    ~results["inventory"]["type"].isin(["TimePeriod", "Trench"])
]

colors1 = plt.cm.Pastel1(range(len(df_filtered)))
bars1 = ax1.bar(
    df_filtered["type"],
    df_filtered["count"],
    color=colors1,
    edgecolor="black",
    linewidth=1.5,
)

ax1.set_title("Feature Type Inventory", fontsize=15, fontweight="bold", pad=10)
ax1.set_ylabel("Count", fontweight="bold", fontsize=13)
plt.setp(ax1.xaxis.get_majorticklabels(), rotation=45, ha="right", fontsize=11)
ax1.grid(axis="y", alpha=0.3, linestyle="--")

# =====
# Panel 2: Stratigraphic Sequence (with logarithmic scale)
# =====

ax2 = fig.add_subplot(gs[0, 1])

df_strat_sorted = results["stratigraphy"].sort_values("count", ascending=True)
colors2 = plt.cm.Set3(range(len(df_strat_sorted)))

y_positions = range(len(df_strat_sorted))
bars2 = ax2.barh(
    y_positions,
    df_strat_sorted["count"],
    color=colors2,
    edgecolor="black",
    linewidth=1.5,
)

ax2.set_yticks(y_positions)
ax2.set_yticklabels(df_strat_sorted["period"], fontsize=12)
ax2.set_title("Stratigraphic Sequence", fontsize=15, fontweight="bold", pad=10)
ax2.set_xlabel("Number of Features (log scale)", fontweight="bold", fontsize=13)

```

```

ax2.set_xscale("log") # Logarithmic scale for wide value ranges
ax2.grid(axis="x", alpha=0.3, linestyle="--", which="both")

# =====
# Panel 3: Spatial Distribution by Trench
# =====

ax3 = fig.add_subplot(gs[1, 0])

colors3 = ["#8dd3c7", "#fb8072"]
x_positions = range(len(results["trench"]))
bars3 = ax3.bar(
    x_positions,
    results["trench"]["count"],
    color=colors3,
    edgecolor="black",
    linewidth=1.5,
)

ax3.set_xticks(x_positions)
ax3.set_xticklabels(results["trench"]["trench"], fontsize=12)
ax3.set_title(
    "Spatial Distribution by Trench", fontsize=15, fontweight="bold", pad=10
)

ax3.set_ylabel("Count", fontweight="bold", fontsize=13)
ax3.grid(axis="y", alpha=0.3, linestyle="--")

# =====
# Panel 4: Preservation State (Pie Chart)
# =====

ax4 = fig.add_subplot(gs[1, 1])

colors4 = ["#8dd3c7", "#fb8072"]
wedges4, texts4, autotexts4 = ax4.pie(
    results["koetbullar"]["count"],
    labels=results["koetbullar"]["condition"],
    autopct="%1.1f%%",
    colors=colors4,
    startangle=90,
    textprops={"fontsize": 13, "weight": "bold"},
)

ax4.set_title(
    "Kötbullar Preservation State", fontsize=15, fontweight="bold", pad=10
)

for autotext in autotexts4:
    autotext.set_color("white")
    autotext.set_fontweight("bold")

```

```

autotext.set_fontsize(12)

# =====
# Overall Title
# =====

fig.suptitle(
    "FAIRyland Archaeological Dataset - Summary Dashboard",
    fontsize=20,
    fontweight="bold",
    y=0.98,
)

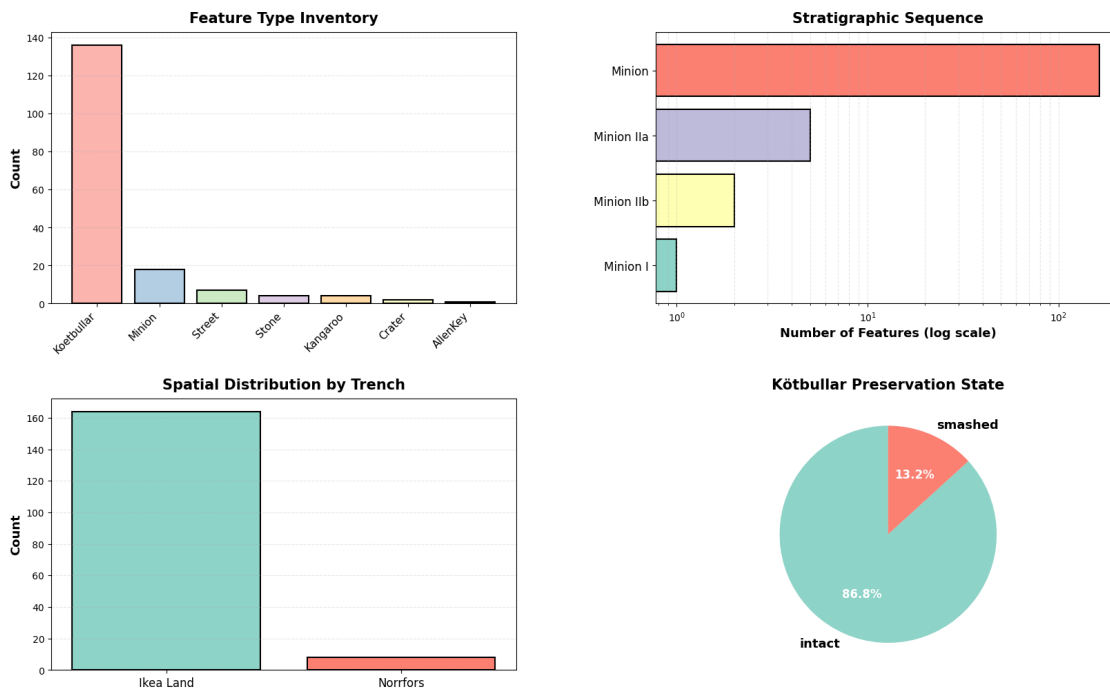
plt.tight_layout()
plt.show()

```

C:\Users\flori\AppData\Local\Temp\ipykernel_24320\711273662.py:107: UserWarning:
This figure includes Axes that are not compatible with tight_layout, so results
might be incorrect.

```
plt.tight_layout()
```

FAIRyland Archaeological Dataset - Summary Dashboard



1.7 Save Dashboard (Optional)

Save the dashboard as high-resolution image.


```
[7]: # Create output directory
out_dir = "out"
os.makedirs(out_dir, exist_ok=True)

# Save figure
output_path = os.path.join(out_dir, "fairyland_summary_dashboard.jpg")

# Recreate figure for saving (previous one was shown)
# ... (repeat visualization code from previous cell) ...
# fig.savefig(output_path, dpi=300, format='jpg', bbox_inches='tight')

print(f"To save the dashboard, re-run the visualization cell and add:")
print(f"  fig.savefig('{output_path}', dpi=300, format='jpg',
    ↪bbox_inches='tight')")
print(f"  before plt.show()")
```

To save the dashboard, re-run the visualization cell and add:

```
    fig.savefig('out\fairyland_summary_dashboard.jpg', dpi=300, format='jpg',
bbox_inches='tight')
    before plt.show()
```